# Array Methods Deep Dive


array methods explain all concept deeply
JavaScript **Array Methods** are functions used to **create, modify, search, iterate, and transform arrays**. I will explain **all important array methods deeply with examples, syntax, behavior, and interview tips.**

## 1. push() – Add element at end

Adds element to the **end of array**.

### Syntax

```
JavaScript



array.push(element1, element2, ...)
```

## Example

```
JavaScript
```

```
let arr = [1,2,3]; arr.push(4); console.log(arr);
```

Output

```
[1,2,3,4]
```

## Important Points

• Changes original array (mutable)
• Returns **new length**

```
JavaScript
```

```
let x = arr.push(5);
console.log(x); // 5
```

Time Complexity → **O(1)**

## 2. pop() – Remove last element

Removes last element from array.

### Syntax

```
JavaScript



array.pop()
```

### Example

```
JavaScript



let arr = [10,20,30]; arr.pop(); console.log(arr);
```

Output

```
[10,20]
```

Returns removed element

```
JavaScript
```

```
let x = arr.pop();
console.log(x);
```

Time Complexity → **O(1)**

## 3. shift() – Remove first element

Removes element from **beginning**.

```
JavaScript
```

```
let arr = [1,2,3]; arr.shift(); console.log(arr);
```

Output

```
[2,3]
```

Time Complexity → **O(n)**
(because elements shift left)

## 4. unshift() – Add element at beginning

```
JavaScript

let arr = [2,3]; arr.unshift(1); console.log(arr);
```

Output

```
[1,2,3]
```

Time Complexity → **O(n)**

## 5. concat() – Merge arrays

Creates **new array**.

```
JavaScript

let a = [1,2];
let b = [3,4]; let c = a.concat(b); console.log(c);
```

Output

```
[1,2,3,4]
```

Does **not modify original array**

## 6. includes() – Check value

```
JavaScript



let arr = [10,20,30]; arr.includes(20);
```

Output

```
true
```

Use for **existence checking**

## 7. indexOf() – Find index

```
JavaScript



let arr = [10,20,30]; arr.indexOf(20);
```

Output

```
1
```

If not found → **-1**

## 8. join() – Convert array to string

```
JavaScript



let arr = ["I","love","JS"]; let str = arr.join(" "); console.log(str);
```

Output

```
I love JS
```

## 9. reverse() – Reverse array

```
JavaScript



let arr = [1,2,3]; arr.reverse(); console.log(arr);
```

Output

```
[3,2,1]
```

Modifies original array.

## 10. slice() – Extract part of array

Does **not modify original array**

### Syntax

```
JavaScript

array.slice(start, end)
```

Example

```
JavaScript

let arr = [1,2,3,4,5]; let part = arr.slice(1,4); console.log(part);
```

Output

```
[2,3,4]
```

## 11. splice() – Insert / delete elements

Most powerful array method.

## Syntax

```
array.splice(start, deleteCount, item1, item2)
```

## Delete

```
JavaScript



let arr = [1,2,3,4]; arr.splice(1,2); console.log(arr);
```

Output

```
[1,4]
```

### Insert

```
JavaScript

let arr = [1,4]; arr.splice(1,0,2,3); console.log(arr);
```

Output

```
[1,2,3,4]
```

## 12. sort()

Sorts elements.

```
JavaScript

let arr = [4,2,7,1]; arr.sort(); console.log(arr);
```

Output

```
[1,2,4,7]
```

## Problem

Numbers sort as strings.

```
[100,20,3]
```

Result

```
[100,20,3]
```

## Correct numeric sort

```
JavaScript
```

```
arr.sort((a,b)=>a-b)
```

## 13. map() – Transform array

Returns **new array**

```
JavaScript
```

```
let nums = [1,2,3]; let result = nums.map(function(n){
    return n * 2;
}); console.log(result);
```

Output

```
[2,4,6]
```

## 14. filter() – Filter values

```
JavaScript
```

```
let nums = [10,25,30,15]; let result = nums.filter(function(n){
    return n > 20;
}); console.log(result);
```

Output

```
[25,30]
```

## 15. reduce() – Reduce to single value

Very important for interviews.

```
JavaScript
```

```
let nums = [1,2,3,4]; let sum = nums.reduce(function(acc, curr){
    return acc + curr;
},0); console.log(sum);
```

Output

```
10
```

Concept

```
accumulator + current
```

## 16. find()

Returns **first matching element**

```
JavaScript



let arr = [5,10,15]; let x = arr.find(n => n > 6); console.log(x);
```

Output

```
10
```

## 17. findIndex()

Returns index.

```
JavaScript
```

```
arr.findIndex(n => n > 6)
```

Output

```
1
```

## 18. every()

Checks if **all elements satisfy condition**

```
JavaScript
```

```
let arr = [10,20,30]; arr.every(n => n > 5);
```

Output

```
true
```

## 19. some()

Checks if **at least one element satisfies condition**

```
JavaScript



arr.some(n => n > 25)
```

Output

```
true
```

## 20. flat()

Flatten nested arrays.

```
JavaScript
```

```
let arr = [1,[2,3],[4,5]]; arr.flat();
```

Output

```
[1,2,3,4,5]
```

## 21. flatMap()

Combination of **map + flat**

```
JavaScript
```

```
let arr = [1,2,3]; arr.flatMap(n => [n,n*2])
```

Output

```
[1,2,2,4,3,6]
```

## Mutable vs Immutable Methods

### Mutable (change original array)

```
push
pop
shift
unshift
splice
sort
reverse
```

### Immutable (return new array)

```
map
filter
slice
concat
flat
reduce
```

# Most Asked Interview Questions

## Q1 Difference between map and forEach?

map

```
returns new array
```

forEach

```
does not return array
```

## Q2 Difference between slice and splice?

slice

```
does not modify original array
```

splice

```
modifies original array
```

## Q3 Which method converts array to single value?

```
reduce()
```

## Real Example

```
JavaScript



let prices = [100,200,300]; let total = prices
            .map(p => p * 0.9)
            .reduce((a,b)=>a+b); console.log(total);
```

If you want, I can also explain 🔥 **25 Most Important JavaScript Array Interview Questions with Deep Explanation and Tricky Examples.**
These are **very commonly asked in interviews.**

// let arr = [1, 2, 3, 4, 5, 6]; // let print = function(ele){ // console.log(ele); // }; // arr.forEach(print); // arr.forEach(function(ele){ // console.log(ele); // }); // output: 1 2 3 4 5 6 // using arrow function let arr = [1, 2, 3, 4, 5, 6]; arr.forEach((ele) => { console.log(ele); }); console.log("other way\n"); // output: 1 2 3 4 5 6 arr.forEach(function (ele){ console.log(ele); }); console.log("other way"); arr.forEach(ele => console.log(ele)); let arr1 = [ { name: "aamir", id: 1, marks: 95, }, { name: "kamran", id: 2, marks: 90, }, { name: "khan", id: 3, marks: 80, }, ]; console.log(arr1); arr1.forEach((ele) => { console.log(ele); }); // Array Methods // • forEach // • map // • filter // • some // • every // • reduce //forEach => forEach is used to iterate over an array and perform a specific operation on each element. let arr2 = [1, 2, 3, 4, 5, 6]; arr2.forEach((ele) => { console.log(ele); }); //map=> map is used to create a new array by performing a specific operation on each element of the original array. let arr3 = [1, 2, 3, 4, 5, 6]; let square = arr3.map((ele) => ele * ele); console.log(square); //filter => it means to filter out elements from an array based on a certain condition. let arr4 = [1, 2, 3, 4, 5, 6]; let even = arr4.filter((ele) => ele % 2 == 0); console.log(even); //some => some is used to check if at least one element in an array

satisfies a certain condition. let arr5 = [1, 2, 3, 4, 5, 6]; let ans = arr5.some((ele) => ele % 2 == 0); console.log(ans); //every => every is used to check if every element in an array satisfies a certain condition. let arr6 = [1, 2, 3, 4, 5, 6]; let ans1 = arr6.every((ele) => ele % 2 == 0); console.log(ans1); //reduce => reduce is used to reduce an array to a single value. let arr7 = [1, 2, 3, 4, 5, 6]; let sum = arr7.reduce((acc, cur) => acc + cur, 0); console.log(sum);

Your code is **correct**, and you used the most important JavaScript array methods. I will now explain **each part of your code deeply** so you understand **how it works internally, callback concept, parameters, and interview points**.

## 1. forEach() Method

### Your Code

```
JavaScript



let arr = [1, 2, 3, 4, 5, 6];
arr.forEach((ele) => {
    console.log(ele);
});
```

### Output

```
1
2
3
4
5
6
```

## Concept

`forEach()` is used to **iterate through every element of an array**.

It **does not return anything**.

## Syntax

```
JavaScript



array.forEach(callbackFunction)
```

## Internal Working

JavaScript internally does something like this:

```
JavaScript



for(let i = 0; i < arr.length; i++){
    callback(arr[i]);
}
```

So it runs the callback function **for every element**.

## 2. Different Ways to Write forEach

### Method 1 (Function Reference)

```
JavaScript




let print = function(ele){
    console.log(ele);
}; arr.forEach(print);
```

Here

```
print → callback function
```

### Method 2 (Normal Function)

```
JavaScript




arr.forEach(function(ele){
    console.log(ele);
});
```

Here function is written **directly inside forEach**.

### Method 3 (Arrow Function)

```
JavaScript




arr.forEach((ele) => {
    console.log(ele);
});
```

Arrow function is **shorter syntax**.

### Method 4 (Shortest Arrow Function)

```
JavaScript




arr.forEach(ele => console.log(ele));
```

When **only one statement exists**, `{}` is optional.

## 3. Important Parameters of forEach

`forEach()` actually gives **3 parameters**.

```
JavaScript
```

```
arr.forEach((element, index, array) => {
    console.log(element, index);
});
```

## Example

```
JavaScript
```

```
arr.forEach((ele, i) => {
    console.log(i, ele);
});
```

## Output

```
0 1
1 2
2 3
3 4
4 5
5 6
```

### Parameters

| Parameter | Meaning |
| --- | --- |
| element | current value |
| index | index number |
| array | original array |

## 4. forEach with Object Array

### Your Code

```
JavaScript




let arr1 = [
    { name: "aamir", id: 1, marks: 95 },
    { name: "kamran", id: 2, marks: 90 },
    { name: "khan", id: 3, marks: 80 },
];
```

### Loop

```
JavaScript




arr1.forEach((ele) => {
    console.log(ele);
});
```

Output

```
{name:"aamir",id:1,marks:95}
{name:"kamran",id:2,marks:90}
{name:"khan",id:3,marks:80}
```

## Access Specific Property

```
JavaScript



arr1.forEach((student)=>{
    console.log(student.name, student.marks);
});
```

Output

```
aamir 95
kamran 90
khan 80
```

## 5. map() Method

### Your Code

```
JavaScript
```

```
let arr3 = [1,2,3,4,5,6];
let square = arr3.map((ele) => ele * ele);
```

### Output

```
[1,4,9,16,25,36]
```

### Concept

`map()` **creates a new array** by transforming each element.

## Internal Working

```
JavaScript
```

```
let newArray = []; for(let i=0;i<arr3.length;i++){
   newArray.push(arr3[i] * arr3[i]);
}
```

## Important Difference

| Method | Return |
| --- | --- |
| forEach | undefined |
| map | new array |

# 6. filter() Method

## Your Code

```
JavaScript
```

```
let even = arr4.filter((ele) => ele % 2 == 0);
```

## Output

```
[2,4,6]
```

## Concept

`filter()` keeps **only elements that satisfy condition**.

## Internal Working

```
JavaScript



let result = []; for(let i=0;i<arr4.length;i++){
    if(arr4[i] % 2 == 0){
        result.push(arr4[i]);
    }
}
```

## 7. some() Method

### Your Code

```
JavaScript



let ans = arr5.some((ele) => ele % 2 == 0);
```

### Output

```
true
```

### Concept

Checks if **at least one element satisfies condition**.

Stops immediately when condition becomes **true**.

Example

```
JavaScript
```

```
[1,3,5,6]
```

It stops when it finds **6**.

## 8. every() Method

### Your Code

```
JavaScript
```

```
let ans1 = arr6.every((ele) => ele % 2 == 0);
```

### Output

```
false
```

### Concept

Checks if **all elements satisfy condition**.

Example

```
[2,4,6] → true
[2,3,6] → false
```

## 9. reduce() Method

### Your Code

```JavaScript
let sum = arr7.reduce((acc, cur) => acc + cur, 0);
```

### Output

```
21
```

### Concept

`reduce()` converts array → **single value**

## Working Step by Step

Array

```
[1,2,3,4,5,6]
```

Initial

```
acc = 0
```

Step 1

```
0 + 1 = 1
```

Step 2

```
1 + 2 = 3
```

Step 3

```
3 + 3 = 6
```

Step 4

```
6 + 4 = 10
```

Step 5

```
10 + 5 = 15
```

Step 6

```
15 + 6 = 21
```

Final Output

```
21
```

## 10. Powerful Real Example (Chain Methods)

```
JavaScript




let numbers = [1,2,3,4,5,6]; let result = numbers
              .filter(n => n % 2 == 0)
              .map(n => n * n)
              .reduce((a,b) => a + b); console.log(result);
```

Step

```
filter → [2,4,6]
map → [4,16,36]
reduce → 56
```

Output

```
56
```

## Most Important Interview Question

### Difference Table

| Method | Returns | Purpose |
|---|---|---|
| forEach | undefined | iterate |
| map | new array | transform |
| filter | new array | filter elements |
| some | boolean | at least one |
| every | boolean | all elements |
| reduce | single value | aggregate |

If you want, I can also explain 🔥 **10 SUPER IMPORTANT Array Method Interview Problems (Amazon / Google level)** like:

1️⃣ flatten array
2️⃣ group objects
3️⃣ remove duplicates
4️⃣ frequency counter
5️⃣ sum of objects
6️⃣ custom map function

These **greatly improve JavaScript mastery.**

map & filter all concept

I'll explain `map()` **and** `filter()` **in JavaScript deeply** — including **syntax, parameters, internal working, differences, chaining, objects usage, interview tricks, and edge cases**.

## 1. map() Method

## Definition

`map()` is used to **create a new array by transforming each element of the original array**.

It **does not change the original array**.

## Syntax

```
JavaScript



array.map(callback(element, index, array))
```

### Parameters

| Parameter | Meaning |
| --- | --- |
| element | current element |
| index | position of element |
| array | original array |

## Basic Example

```
JavaScript



let nums = [1,2,3,4]; let result = nums.map((n)=>{
    return n*2;
}); console.log(result);
```

Output

```
[2,4,6,8]
```

Original array remains same

```
[1,2,3,4]
```

## 2. Internal Working of map()

JavaScript internally works like this:

```
JavaScript
```

```
let newArray = []; for(let i=0;i<arr.length;i++){
    newArray.push(callback(arr[i]));
}
```

So it:

1. Iterates array
2. Applies function
3. Stores result in new array

## 3. Short Arrow Syntax

```
JavaScript



let nums = [1,2,3]; let square = nums.map(n => n*n); console.log(square);
```

Output

```
[1,4,9]
```

## 4. Using index in map()

```
JavaScript



let nums = [10,20,30]; let result = nums.map((value,index)=>{
    return value + index;
}); console.log(result);
```

Output

```
[10,21,32]
```

## 5. map() with Objects

Example array

```
JavaScript
```

```
let students = [
 {name:"aamir", marks:95},
 {name:"kamran", marks:90},
 {name:"khan", marks:80}
];
```

## Extract names

```
JavaScript
```

```
let names = students.map(student => student.name); console.log(names);
```

Output

```
["aamir","kamran","khan"]
```

## 6. Transform Object Data

```
JavaScript
```

```
let result = students.map(student => {
    return {
        name: student.name,
        result: student.marks > 90 ? "Topper" : "Pass"
    };
}); console.log(result);
```

Output

```
[
 {name:"aamir", result:"Topper"},
 {name:"kamran", result:"Pass"},
 {name:"khan", result:"Pass"}
]
```

## 7. Important map() Characteristics

✔ Returns **new array**
✔ Same length as original array
✔ Does not modify original array

## 8. filter() Method

## Definition

`filter()` creates a **new array containing elements that satisfy a condition**.

## Syntax

```
JavaScript



array.filter(callback(element,index,array))
```

## 9. Basic Example

```
JavaScript



let nums = [1,2,3,4,5,6]; let even = nums.filter(n => n % 2 === 0); console.log(even);
```

Output

```
[2,4,6]
```

## 10. Internal Working of filter()

JavaScript internally works like:

```
JavaScript
```

```
let result = []; for(let i=0;i<arr.length;i++){
    if(callback(arr[i])){
        result.push(arr[i]);
    }
}
```

So it only **keeps elements where condition = true**.

## 11. Filter with Objects

Example

```
JavaScript
```

```
let students = [
 {name:"aamir", marks:95},
 {name:"kamran", marks:90},
 {name:"khan", marks:80}
];
```

## Filter toppers

```
JavaScript
```

```
let toppers = students.filter(student => student.marks > 90); console.log(toppers);
```

Output

```
[
 {name:"aamir", marks:95}
]
```

## 12. Multiple Conditions

```
JavaScript



let nums = [10,20,30,40,50]; let result = nums.filter(n => n > 20 && n <
50); console.log(result);
```

Output

```
[30,40]
```

## 13. Important filter() Characteristics

✔ Returns **new array**
✔ Length may be **smaller**
✔ Does not modify original array

## 14. map() vs filter()

| Feature | map | filter |
| --- | --- | --- |
| Purpose | transform elements | remove elements |
| Output length | same | smaller or same |
| Return type | new array | new array |

## Example

```
JavaScript



let nums = [1,2,3,4];
```

map

```
JavaScript



nums.map(n=>n*2)
```

Output

```
[2,4,6,8]
```

filter

```
JavaScript



nums.filter(n=>n>2)
```

Output

```
[3,4]
```

## 15. Powerful Concept: Method Chaining

You can combine methods.

Example

```
JavaScript



let nums = [1,2,3,4,5,6]; let result = nums
            .filter(n => n % 2 === 0)
            .map(n => n * n); console.log(result);
```

Step

```
filter → [2,4,6]
map → [4,16,36]
```

Output

```
[4,16,36]
```

## 16. Real World Example

E-commerce prices

```
JavaScript
```

```
let prices = [100,200,300,400]; let discounted = prices
                .filter(price => price > 200)
                .map(price => price * 0.9); console.log(discounted);
```

Output

```
[270,360]
```

## 17. Interview Trick Question

### What happens if map() returns nothing?

```
JavaScript



let arr = [1,2,3]; let result = arr.map(n => {}); console.log(result);
```

Output

```
[undefined, undefined, undefined]
```

Because map **always returns array of same length**.

## 18. Another Trick

filter returning non-boolean still works.

```
JavaScript
```

```
let arr = [0,1,2,3]; let result = arr.filter(n => n); console.log(result);
```

Output

```
[1,2,3]
```

Because `0` is **falsey**.

## 19. Common Interview Question

### Remove duplicate numbers

```
JavaScript
```

```
let nums = [1,2,2,3,4,4]; let unique = nums.filter((value,index,array)=>{
    return array.indexOf(value) === index;
}); console.log(unique);
```

Output

```
[1,2,3,4]
```

## 20. Advanced Example

Objects filtering + mapping

```
JavaScript



let students = [
 {name:"aamir",marks:95},
 {name:"kamran",marks:90},
 {name:"khan",marks:80}
]; let result = students
             .filter(s => s.marks > 85)
             .map(s => s.name); console.log(result);
```

Output

```
["aamir","kamran"]
```

If you want, I can also explain 🔥 **15 Advanced JavaScript Array Problems (map/filter/reduce)** that are **frequently asked in interviews and React development**.
every and sum
I will explain `every()` **and** `sum using reduce()` **deeply** because in JavaScript **sum is usually calculated using** `reduce()`.

# 1. every() Method

## Definition

`every()` checks **whether all elements in the array satisfy a condition**.

If **all elements satisfy → true**
If **one element fails → false**

## Syntax

```
JavaScript



array.every(callback(element, index, array))
```

### Parameters

| Parameter | Meaning |
| --- | --- |
| element | current element |
| index | element index |
| array | original array |

## Basic Example

```
JavaScript



let arr = [2,4,6,8]; let result = arr.every(num => num % 2 === 0); console.log(result);
```

Output

```
true
```

Because **all numbers are even**.

## Example 2

```
JavaScript



let arr = [2,4,5,8]; let result = arr.every(num => num % 2 === 0); console.log(result);
```

Output

```
false
```

Because **5 is odd**.

## Internal Working of every()

JavaScript internally works like this:

```
JavaScript
```

```
for(let i = 0; i < arr.length; i++){
    if(!condition){
        return false;
    }
} return true;
```

It **stops immediately** when condition becomes false.

## Example with Index

```
JavaScript
```

```
let arr = [10,20,30]; arr.every((value,index)=>{
    console.log(value,index);
    return value > 5;
});
```

Output

```
10 0
20 1
30 2
true
```

## every() with Objects

```
JavaScript
```

```
let students = [
 {name:"aamir", marks:95},
 {name:"kamran", marks:90},
 {name:"khan", marks:80}
]; let result = students.every(s => s.marks > 70); console.log(result);
```

Output

```
true
```

## Difference between every() and some()

| Method | Meaning |
| --- | --- |
| every() | all elements must satisfy condition |
| some() | at least one element must satisfy |

Example

```
JavaScript



let arr = [2,4,6]; arr.some(n => n % 2 !== 0);
```

Output

```
false
```

## 2. Sum of Array (Using reduce)

JavaScript **does not have a direct sum() method**.
We use **reduce()**.

## reduce() Concept

`reduce()` converts **array → single value**.

Examples

- sum
- product
- average
- max value

## Syntax

```
JavaScript
```

```
array.reduce((accumulator, currentValue) => {}, initialValue)
```

## Parameters

| Parameter | Meaning |
| --- | --- |
| accumulator | previous result |
| currentValue | current element |

## Example: Sum of Array

```
JavaScript
```

```
let arr = [1,2,3,4,5]; let sum = arr.reduce((acc, cur) => acc + cur,
0); console.log(sum);
```

Output

```
15
```

## Step by Step Working

Array

```
[1,2,3,4,5]
```

Initial value

```
acc = 0
```

Step 1

```
0 + 1 = 1
```

Step 2

```
1 + 2 = 3
```

Step 3

```
3 + 3 = 6
```

Step 4

```
6 + 4 = 10
```

Step 5

```
10 + 5 = 15
```

Final Output

```
15
```

## Example Without Initial Value

```
JavaScript
```

```
let arr = [1,2,3,4]; let sum = arr.reduce((acc,cur)=>acc+cur); console.log(sum);
```

Here

```
acc = first element
cur = second element
```

Working

```
1+2=3
3+3=6
6+4=10
```

Output

```
10
```

## Sum of Object Values

Example

```
JavaScript
```

```
let items = [
 {price:100},
 {price:200},
 {price:300}
]; let total = items.reduce((sum,item)=>{
    return sum + item.price;
},0); console.log(total);
```

Output

```
600
```

## Powerful Example (Filter + Reduce)

```
JavaScript




let nums = [1,2,3,4,5,6]; let result = nums
             .filter(n => n % 2 === 0)
             .reduce((sum,n)=> sum + n ,0); console.log(result);
```

### Step

```
filter → [2,4,6]
sum → 12
```

### Output

```
12
```

## Interview Difference

| Method | Return | Purpose |
| --- | --- | --- |
| every() | boolean | check all elements |
| reduce() | single value | combine array |

## Important Trick Question

### What happens with empty array?

```
JavaScript



[].every(x => x > 10)
```

Output

```
true
```

Because **no element violates the condition**.

If you want, I can also explain 🔥 **reduce() Advanced Concepts** like:

• flatten array
• group objects
• frequency counter
• custom map/filter using reduce

These are **very important for JavaScript interviews and React development.**

reduce method

The `reduce()` **method** in JavaScript is one of the **most powerful array methods**. It is used to **convert an entire array into a single value** (number, object, array, etc.).

Examples of what `reduce()` can do:

- Calculate **sum**
- Find **maximum / minimum**
- **Flatten arrays**
- **Count frequency**
- **Group objects**
- Implement **map and filter**

Let's understand it **step by step deeply**.

## 1. Definition

`reduce()` executes a **callback function on each element of the array** and accumulates the result into **one final value**.

## 2. Syntax

```
JavaScript



array.reduce(callback, initialValue)
```

Callback function:

```
JavaScript

(accumulator, currentValue, index, array) => {}
```

### Parameters

| Parameter | Meaning |
| --- | --- |
| accumulator | Stores the previous result |
| currentValue | Current element |
| index | Current index |
| array | Original array |
| initialValue | Starting value of accumulator |

## 3. Basic Example (Sum)

```
JavaScript

let arr = [1,2,3,4,5]; let sum = arr.reduce((acc, cur) => {
    return acc + cur;
}, 0); console.log(sum);
```

Output

```
15
```

## 4. Step-by-Step Working

Array

```
[1,2,3,4,5]
```

Initial value

```
acc = 0
```

Iterations

```
0 + 1 = 1
1 + 2 = 3
3 + 3 = 6
6 + 4 = 10
10 + 5 = 15
```

Final result

```
15
```

## 5. Reduce Without Initial Value

```
JavaScript
```

```
let arr = [1,2,3,4]; let sum = arr.reduce((acc, cur) => acc + cur); console.log(sum);
```

Here:

```
acc = first element
cur = second element
```

Steps

```
1 + 2 = 3
3 + 3 = 6
6 + 4 = 10
```

Output

```
10
```

## 6. Find Maximum Value

```
JavaScript
```

```
let nums = [5,10,3,8]; let max = nums.reduce((acc, cur)=>{
    return acc > cur ? acc : cur;
}); console.log(max);
```

Output

```
10
```

## 7. Find Minimum Value

```
JavaScript
```

```
let nums = [5,10,3,8]; let min = nums.reduce((acc, cur)=>{
    return acc < cur ? acc : cur;
}); console.log(min);
```

Output

```
3
```

## 8. Count Frequency of Elements

Example

```
JavaScript
```

```
let arr = ["a","b","a","c","b","a"]; let freq = arr.reduce((acc, cur)=>{
if(acc[cur]){
        acc[cur]++;
    }else{
        acc[cur] = 1;
    }     return acc; }, {}); console.log(freq);
```

Output

```
{a:3, b:2, c:1}
```

## 9. Flatten Array

```
JavaScript
```

```
let arr = [[1,2],[3,4],[5,6]]; let flat = arr.reduce((acc,cur)=>{
    return acc.concat(cur);
},[]); console.log(flat);
```

Output

```
[1,2,3,4,5,6]
```

## 10. Sum of Object Values

```
JavaScript
```

```
let items = [
 {name:"pen", price:10},
 {name:"book", price:50},
 {name:"bag", price:100}
]; let total = items.reduce((sum,item)=>{
    return sum + item.price;
},0); console.log(total);
```

Output

```
160
```

## 11. Reduce with Filter Concept

Example: Sum of even numbers

```
JavaScript
```

```
let nums = [1,2,3,4,5,6]; let sum = nums.reduce((acc,cur)=>{

    if(cur % 2 === 0){
        acc += cur;
    }     return acc; },0); console.log(sum);
```

Output

```
12
```

## 12. Implement map() using reduce()

```
JavaScript
```

```
let nums = [1,2,3]; let result = nums.reduce((acc,cur)=>{
    acc.push(cur*2);
    return acc;
},[]); console.log(result);
```

Output

```
[2,4,6]
```

## 13. Implement filter() using reduce()

```
JavaScript
```

```
let nums = [1,2,3,4,5,6]; let even = nums.reduce((acc,cur)=>{

    if(cur % 2 === 0){
        acc.push(cur);
    }     return acc; },[]); console.log(even);
```

Output

```
[2,4,6]
```

## 14. Real World Example

Calculate **total cart price**

```
JavaScript
```

```
let cart = [
 {product:"phone", price:20000},
 {product:"earbuds", price:3000},
 {product:"charger", price:1000}
]; let total = cart.reduce((sum,item)=>sum + item.price,0); console.log(total);
```

Output

```
24000
```

## 15. Key Characteristics

✔ Returns **single value**
✔ Very **powerful and flexible**
✔ Can replace **map, filter, find, sum**

## 16. Interview Difference

| Method | Output |
|---|---|
| map | new array |
| filter | filtered array |
| reduce | single value |

## 17. Common Interview Question

What happens if reduce() runs on empty array without initial value?

```
JavaScript



[].reduce((a,b)=>a+b)
```

Result

```
Error
```

Because **there is no initial accumulator value**.

If you want, I can also explain 🔥 **10 Advanced reduce() Interview Problems** like:

- groupBy using reduce
- remove duplicates using reduce
- nested array flattening
- shopping cart calculation
- counting objects

These are **very common in React and JavaScript interviews.**

maximum in array

To find the **maximum value in an array in JavaScript**, there are **multiple methods**. I will explain them **step-by-step from basic to advanced**.

## 1. Using reduce() (Most Important)

This is **very common in interviews**.

## Code

```
JavaScript
```

```
let arr = [5, 10, 3, 8, 20]; let max = arr.reduce((acc, cur) => {
    return acc > cur ? acc : cur;
}); console.log(max);
```

## Output

```
20
```

## Step-by-step

Array

```
[5,10,3,8,20]
```

Steps

```
5 > 10 → 10
10 > 3 → 10
10 > 8 → 10
10 > 20 → 20
```

Final result

```
20
```

## 2. Using Math.max()

### Code

```JavaScript
let arr = [5, 10, 3, 8, 20]; let max = Math.max(...arr); console.log(max);
```

## Output

```
20
```

## Why `...` (spread operator)?

`Math.max()` expects numbers like this:

```
Math.max(5,10,3,8,20)
```

But array is:

```
[5,10,3,8,20]
```

Spread operator converts it.

## 3. Using for Loop (Basic Logic)

```
JavaScript
```

```
let arr = [5,10,3,8,20]; let max = arr[0]; for(let i=1;i<arr.length;i++){     if(arr[i]
> max){
        max = arr[i];
    } } console.log(max);
```

Output

```
20
```

## 4. Using forEach()

```
JavaScript
```

```
let arr = [5,10,3,8,20]; let max = arr[0]; arr.forEach((num)=>{
    if(num > max){
        max = num;
    }
}); console.log(max);
```

Output

```
20
```

## 5. Maximum in Array of Objects

Example

```
JavaScript
```

```
let students = [
 {name:"Aamir", marks:95},
 {name:"Kamran", marks:80},
 {name:"Khan", marks:90}
];
```

## Find highest marks

```
JavaScript
```

```
let maxMarks = students.reduce((max, student)=>{     return student.marks > max ?
student.marks : max; },0); console.log(maxMarks);
```

Output

```
95
```

## 6. Find Object with Maximum Value

```
JavaScript
```

```
let topper = students.reduce((max, student)=>{     return student.marks > max.marks ? student : max; }); console.log(topper);
```

Output

```
{name:"Aamir", marks:95}
```

## 7. Interview Trick

What happens if array is empty?

```
JavaScript
```

```
Math.max(...[])
```

Output

```
-Infinity
```

## 8. Important Interview Comparison

| Method | Use |
| --- | --- |
| reduce() | best for interviews |
| Math.max(...arr) | shortest |
| for loop | basic logic |

### ✅ Best Practice

```
Math.max(...arr)
```

or

```
reduce()
```

If you want, I can also show 🔥 **10 advanced array interview questions** like:

- second largest number
- remove duplicates
- find missing number
- rotate array
- flatten array

These are **very common JavaScript interview problems**.

reduce methods

The `reduce()` **method** in JavaScript is one of the **most powerful array methods**. It is used to **reduce an array into a single value**.

That value can be:

- Number (sum, product)
- Object
- Array
- String

## 1. Definition

`reduce()` executes a **callback function on each element of the array** and produces **one final result**.

## 2. Syntax

```
JavaScript


array.reduce(callback, initialValue)
```

Callback parameters

```
JavaScript


(accumulator, currentValue, index, array)
```

## Parameters

| Parameter | Meaning |
| --- | --- |
| accumulator | stores previous result |
| currentValue | current element |
| index | current index |
| array | original array |
| initialValue | starting value |

## 3. Basic Example (Sum)

```
JavaScript
```

```
let arr = [1,2,3,4,5]; let sum = arr.reduce((acc,cur)=>{
    return acc + cur;
},0); console.log(sum);
```

Output

```
15
```

## 4. Step-by-Step Working

Array

```
[1,2,3,4,5]
```

Initial value

```
acc = 0
```

## Steps

```
0 + 1 = 1
1 + 2 = 3
3 + 3 = 6
6 + 4 = 10
10 + 5 = 15
```

## Final Output

```
15
```

## 5. Reduce Without Initial Value

```
JavaScript



let arr = [1,2,3,4]; let sum = arr.reduce((acc,cur)=>acc+cur); console.log(sum);
```

Working

```
1+2=3
3+3=6
6+4=10
```

Output

```
10
```

## 6. Find Maximum Value

```
JavaScript
```

```
let nums = [5,10,3,8]; let max = nums.reduce((acc,cur)=>{
    return acc > cur ? acc : cur;
}); console.log(max);
```

Output

```
10
```

## 7. Find Minimum Value

```
JavaScript
```

```
let nums = [5,10,3,8]; let min = nums.reduce((acc,cur)=>{
    return acc < cur ? acc : cur;
}); console.log(min);
```

Output

```
3
```

## 8. Multiply All Numbers

```
JavaScript
```

```
let arr = [1,2,3,4]; let product = arr.reduce((acc,cur)=>{
    return acc * cur;
},1); console.log(product);
```

Output

```
24
```

## 9. Count Frequency

```
JavaScript




let arr = ["a","b","a","c","b","a"]; let freq = arr.reduce((acc,cur)=>{
if(acc[cur]){
        acc[cur]++;
    }else{
        acc[cur] = 1;
    }     return acc; },{}); console.log(freq);
```

Output

```
{a:3, b:2, c:1}
```

## 10. Flatten Array

```
JavaScript




let arr = [[1,2],[3,4],[5,6]]; let flat = arr.reduce((acc,cur)=>{
    return acc.concat(cur);
},[]); console.log(flat);
```

Output

```
[1,2,3,4,5,6]
```

## 11. Sum of Object Values

```
JavaScript
```

```
let items = [
 {name:"pen",price:10},
 {name:"book",price:50},
 {name:"bag",price:100}
]; let total = items.reduce((sum,item)=>{
    return sum + item.price;
},0); console.log(total);
```

Output

```
160
```

## 12. Reduce with Condition

Sum of even numbers

```
JavaScript
```

```
let nums = [1,2,3,4,5,6]; let result = nums.reduce((acc,cur)=>{     if(cur % 2 === 0){
        acc += cur;
    }     return acc; },0); console.log(result);
```

Output

```
12
```

## 13. Implement map() using reduce()

```
JavaScript
```

```
let nums = [1,2,3]; let result = nums.reduce((acc,cur)=>{
    acc.push(cur*2);
    return acc;
},[]); console.log(result);
```

Output

```
[2,4,6]
```

## 14. Implement filter() using reduce()

```
JavaScript
```

```
let nums = [1,2,3,4,5,6]; let even = nums.reduce((acc,cur)=>{

    if(cur % 2 === 0){
        acc.push(cur);
    }     return acc; },[]); console.log(even);
```

Output

```
[2,4,6]
```

## 15. Real World Example

Shopping cart total

```
JavaScript
```

```
let cart = [
 {product:"phone",price:20000},
 {product:"charger",price:1000},
 {product:"earbuds",price:3000}
]; let total = cart.reduce((sum,item)=>sum+item.price,0); console.log(total);
```

Output

```
24000
```

## 16. Important Points

✔ Returns **single value**
✔ Very **flexible**
✔ Can replace **map, filter, find**

## 17. Common Interview Question

What happens with empty array?

```
JavaScript



[].reduce((a,b)=>a+b)
```

Result

```
Error
```

Because there is **no initial value**.

If you want, I can also explain 🔥 **Advanced reduce() concepts** like:

- groupBy using reduce
- remove duplicates using reduce
- flatten nested arrays
- second largest number
- most frequent element

These are **very important for JavaScript interviews and React development**.

callback function

A **callback function** is a **function passed as an argument to another function**, and it is **executed later by that function**.

This concept is **very important in JavaScript**, especially in:

- Array methods (`map`, `filter`, `reduce`, `forEach`)
- `setTimeout`

- `setInterval`
- Event handling
- Promises
- Async programming

## 1. Basic Definition

A **callback function** is a function that is **passed into another function and executed inside that function**.

## 2. Simple Example

```
JavaScript




function greet(name, callback){
    console.log("Hello " + name);
    callback();
} function sayBye(){
    console.log("Goodbye!");
} greet("Aamir", sayBye);
```

Output

```
Hello Aamir
Goodbye!
```

## Explanation

```
greet() → main function
sayBye() → callback function
```

`sayBye` is passed as an argument and executed inside `greet`.

## 3. Callback Using Anonymous Function

```
JavaScript



function greet(name, callback){
    console.log("Hello " + name);
    callback();
} greet("Aamir", function(){
    console.log("Welcome!");
});
```

Output

```
Hello Aamir
Welcome!
```

## 4. Callback Using Arrow Function

```
JavaScript
```

```
function greet(name, callback){
    console.log("Hello " + name);
    callback();
} greet("Aamir", () => {
    console.log("Welcome!");
});
```

## 5. Callback in Array Methods

Example with **forEach**

```
JavaScript
```

```
let arr = [1,2,3,4]; arr.forEach(function(num){
    console.log(num);
});
```

Here

```
function(num) { console.log(num); }
```

is the **callback function**.

## 6. Callback in map()

```
JavaScript



let nums = [1,2,3]; let result = nums.map(function(num){
    return num * 2;
}); console.log(result);
```

Output

```
[2,4,6]
```

The function inside `map()` is a **callback function**.

## 7. Callback in setTimeout()

```
JavaScript




setTimeout(function(){
    console.log("Hello after 2 seconds");
},2000);
```

Here

```
function(){ console.log("Hello after 2 seconds"); }
```

is the callback.

The browser executes it **after 2 seconds**.

## 8. Why Callbacks Are Used

Callbacks allow:

✔ Code **execution after a task finishes**
✔ **Asynchronous programming**
✔ **Reusable logic**

Example:

```
download file → callback runs
API call → callback runs
timer → callback runs
```

## 9. Synchronous vs Asynchronous Callback

## Synchronous Callback

Runs immediately.

Example

```
JavaScript
```

```
let arr = [1,2,3]; arr.forEach((n)=>{
    console.log(n);
});
```

## Asynchronous Callback

Runs later.

Example

```
JavaScript




setTimeout(()=>{
    console.log("Hello");
},2000);
```

## 10. Real Life Analogy

Imagine ordering food.

```
Order food → restaurant prepares → callback when ready
```

Code version:

```
orderFood(callback)
```

## 11. Callback Function Structure

General structure

```
JavaScript




function mainFunction(callback){
    // some work     callback();
}
```

## 12. Interview Question

### What is callback hell?

When many callbacks are nested.

Example

```
JavaScript




login(function(){
    getUser(function(){
        getPosts(function(){
            getComments(function(){             });
        });
    });
});
```

This becomes **hard to read and maintain**.

Solution:

- Promises
- Async/Await

## 13. Important Points

✔ Functions are **first-class citizens in JavaScript**
✔ Functions can be **passed as arguments**
✔ Callbacks enable **asynchronous programming**

If you want, I can also explain **very important JavaScript interview topics related to callbacks**:

- **Higher Order Functions**
- **Callback Hell**
- **Promises**
- **Async / Await**
- **Event Loop**

These are **core concepts every JavaScript developer must know**.
practice
Good 👍 Since you are learning **array methods and callbacks**, here are **practice questions** from **easy → medium → interview level**. Try to solve them first, then check the solution.

## 1. Print All Elements (forEach)

### Question

Print all elements of an array.

```
JavaScript



let arr = [10, 20, 30, 40];
```

## Solution

```
JavaScript




arr.forEach((ele)=>{
    console.log(ele);
});
```

Output

```
10
20
30
40
```

# 2. Square Each Number (map)

## Question

Create a new array containing the **square of each number**.

```
JavaScript



let nums = [1,2,3,4];
```

## Solution

```
JavaScript



let square = nums.map((n)=>{
    return n*n;
}); console.log(square);
```

Output

```
[1,4,9,16]
```

# 3. Filter Even Numbers (filter)

## Question

Return only **even numbers**.

```
JavaScript



let nums = [1,2,3,4,5,6];
```

## Solution

```
JavaScript



let even = nums.filter((n)=>{
    return n % 2 === 0;
}); console.log(even);
```

### Output

```
[2,4,6]
```

## 4. Check If Any Number > 10 (some)

### Question

```
JavaScript



let nums = [2,4,6,12];
```

Check if **any number is greater than 10**.

### Solution

```
JavaScript



let result = nums.some((n)=>{
    return n > 10;
}); console.log(result);
```

Output

```
true
```

## 5. Check If All Numbers Positive (every)

### Question

```
JavaScript



let nums = [1,3,5,7];
```

Check if **all numbers are positive**.

### Solution

```
JavaScript



let result = nums.every((n)=>{
    return n > 0;
}); console.log(result);
```

Output

```
true
```

## 6. Sum of Numbers (reduce)

### Question

```
JavaScript



let nums = [1,2,3,4,5];
```

Find **sum of numbers**.

### Solution

```
JavaScript



let sum = nums.reduce((acc,cur)=>{
    return acc + cur;
},0); console.log(sum);
```

Output

```
15
```

## 7. Find Maximum Number (reduce)

### Question

```
JavaScript



let nums = [5,9,2,15,7];
```

Find **maximum value**.

### Solution

```
JavaScript



let max = nums.reduce((acc,cur)=>{
    return acc > cur ? acc : cur;
}); console.log(max);
```

Output

```
15
```

## 8. Double Numbers (map)

### Question

```
JavaScript



let nums = [2,4,6];
```

Create array with **double values**.

### Solution

```
JavaScript



let result = nums.map((n)=>{
    return n * 2;
}); console.log(result);
```

Output

```
[4,8,12]
```

## 9. Filter Numbers Greater Than 10

### Question

```
JavaScript



let nums = [5,12,8,20,3];
```

Return numbers **greater than 10**.

### Solution

```
JavaScript



let result = nums.filter((n)=>{
    return n > 10;
}); console.log(result);
```

Output

```
[12,20]
```

## 10. Sum of Even Numbers

### Question

```
JavaScript



let nums = [1,2,3,4,5,6];
```

Find **sum of even numbers**.

### Solution

```
JavaScript



let result = nums
            .filter((n)=> n % 2 === 0)
            .reduce((acc,cur)=> acc + cur ,0); console.log(result);
```

Output

```
12
```

# 11. Array of Objects Practice

## Question

```
JavaScript
```

```
let students = [
 {name:"Aamir", marks:95},
 {name:"Kamran", marks:80},
 {name:"Khan", marks:90}
];
```

Get **only student names**.

## Solution

```
JavaScript
```

```
let names = students.map((s)=>{
    return s.name;
}); console.log(names);
```

Output

```
["Aamir","Kamran","Khan"]
```

## 12. Filter Students with Marks > 85

### Solution

```
JavaScript



let result = students.filter((s)=>{
    return s.marks > 85;
}); console.log(result);
```

## 🔥 Challenge Practice (Very Important)

Try yourself.

### 1

Find **sum of odd numbers**

```
JavaScript



[1,2,3,4,5,6,7]
```

## 2

Find **largest number**

```
JavaScript



[12,45,3,89,22]
```

## 3

Convert this

```
[1,2,3]
```

into

```
[10,20,30]
```

using `map`.

### 4

Check if **all numbers are even**

```
[2,4,6,8]
```

### 5

Remove numbers **less than 5**

```
[1,3,5,7,9]
```

If you want, I can also give 🔥 **20 JavaScript Array Interview Questions (VERY IMPORTANT)** that companies ask for **React / JavaScript developers**.

default parameters

## 1️⃣ Default Parameters in JavaScript

**Default parameters** allow a function parameter to have a **default value** if the user does not pass an argument.

Before ES6, developers used manual checks. Now JavaScript supports **direct default values** in the function definition.

## 1️⃣ Basic Syntax

```
JavaScript
```

```
function greet(name = "Guest") {
    console.log("Hello " + name);
} greet("Aamir");
greet();
```

## Output

```
Hello Aamir
Hello Guest
```

✔ If argument is missing → default value used
✔ If argument is provided → default ignored

## 2️⃣ Dry Run

```
JavaScript




function add(a = 10, b = 5){
    console.log(a + b);
} add(2,3);
add(2);
add();
```

## Execution

```
add(2,3) → a=2 b=3 → 5
add(2) → a=2 b=5 → 7
add() → a=10 b=5 → 15
```

## 3️⃣ Default with Arrow Function

```
JavaScript
```

```
const multiply = (a = 1, b = 1) => a * b; console.log(multiply(5,2));
console.log(multiply(5));
console.log(multiply());
```

Output

```
10
5
1
```

## 4️⃣ Default Using Expressions

Default values can be **expressions**.

```
JavaScript




function area(l = 5, w = 10){
    return l * w;
} console.log(area());
console.log(area(4));
console.log(area(4,6));
```

## Output

```
50
40
24
```

## 5️⃣ Default Using Function

```
JavaScript
```

```
function getValue(){
    return 100;
} function test(a = getValue()){
    console.log(a);
} test(50);
test();
```

Output

```
50
100
```

## 6️⃣ Default Parameter with Array

```
JavaScript




function printArr(arr = []){
    console.log(arr);
} printArr([1,2,3]);
printArr();
```

Output

```
[1,2,3]
[]
```

## 7️⃣ Default Parameter with Object

```
JavaScript




function user(info = {name:"guest", age:0}){
    console.log(info);
} user({name:"Aamir", age:22});
user();
```

Output

```
{name:"Aamir", age:22}
{name:"guest", age:0}
```

## 8️⃣ Default Parameter Using Previous Parameter

You can use **previous parameter values**.

```
JavaScript



function sum(a, b = a){
    console.log(a + b);
} sum(5,3);
sum(5);
```

Output

```
8
10
```

Because

```
b = a
```

## 9️⃣ Undefined vs Null

Default values only work when argument is **undefined**.

```
JavaScript



function test(a = 10){
    console.log(a);
} test(undefined);
test(null);
```

Output

```
10
null
```

✔ `undefined` → default used
❌ `null` → default NOT used

## 🔟 Old Way (Before Default Parameters)

```
JavaScript
```

```
function greet(name){
    name = name || "Guest";
    console.log(name);
}
```

Problem:

```
greet("")
greet(0)
```

These also become `"Guest"` unintentionally.

Default parameters fix this.

## 1️⃣1️⃣ Multiple Default Parameters

```
JavaScript




function student(name="Unknown", marks=0){
    console.log(name, marks);
} student("Aamir",95);
student("Aamir");
student();
```

Output

```
Aamir 95
Aamir 0
Unknown 0
```

## 1️⃣2️⃣ Real Interview Example

```
JavaScript




function calculatePrice(price, tax = 0.18){
    return price + price * tax;
} console.log(calculatePrice(100));
console.log(calculatePrice(100,0.05));
```

Output

```
118
105
```

## ⚡ Most Asked Interview Questions

### 1️⃣ What are default parameters?

Default parameters allow **function parameters to have default values when no argument is passed.**

### 2️⃣ When are default parameters used?

When arguments are **missing or undefined.**

### 3️⃣ Do default parameters work with `null`?

❌ No

They only work with **undefined**.

## 4️⃣ Can default parameters depend on other parameters?

✔ Yes

```
JavaScript
function test(a, b = a*2){
    console.log(a,b);
}
```

# 🚀 Practice Questions

## Q1

```
JavaScript
function greet(name = "Guest"){
 console.log("Hello", name);
} greet();
greet("Aamir");
```

## Q2

Print square using default parameter.

## Q3

Create a function:

```
price = 100
tax = 10%
```

Return total price.

## Q4

Find sum using default parameters.

```
sum(5)
sum(5,10)
```

If you want, I can also explain **VERY IMPORTANT advanced array methods** that JavaScript interviews ask:

- `find()`
- `findIndex()`
- `flat()`
- `flatMap()`
- `sort() deep concept`
- `splice()`
- `slice()`
- `reduce advanced patterns`

These are 🔥 **most important for interviews.**
spread all concept

## Spread Operator (. . .) in JavaScript — Complete Concept

The **spread operator (. . .)** expands **iterable values** (array, string, object) into **individual elements**.

It is mainly used for:

1. Copying arrays
2. Merging arrays
3. Passing arguments to functions
4. Copying objects
5. Merging objects
6. Updating objects/arrays immutably

## 1️⃣ Basic Syntax

```
JavaScript


...
```

Example:

```
JavaScript


let arr = [1,2,3]; console.log(...arr);
```

Output

```
1 2 3
```

Spread **expands the array into separate values**.

## 2️⃣ Spread with Function Arguments

Instead of passing array manually:

```
JavaScript



function sum(a,b,c){
    console.log(a+b+c);
} let arr = [10,20,30]; sum(...arr);
```

Output

```
60
```

Without spread you would do:

```
JavaScript




sum(arr[0], arr[1], arr[2]);
```

Spread makes it easier.

## 3️⃣ Copying an Array

```
JavaScript




let arr1 = [1,2,3];
let arr2 = [...arr1]; console.log(arr2);
```

Output

```
[1,2,3]
```

Important:

```
JavaScript
```

```
arr2.push(4);
console.log(arr1);
console.log(arr2);
```

Output

```
[1,2,3]
[1,2,3,4]
```

Because spread creates a **shallow copy**.

## 4️⃣ Merging Arrays

```
JavaScript
```

```
let arr1 = [1,2,3];
let arr2 = [4,5,6]; let merged = [...arr1, ...arr2]; console.log(merged);
```

Output

```
[1,2,3,4,5,6]
```

## 5️⃣ Adding Elements while Copying

```
JavaScript



let arr = [2,3,4]; let newArr = [1, ...arr, 5]; console.log(newArr);
```

Output

```
[1,2,3,4,5]
```

## 6️⃣ Spread with Strings

Strings are iterable.

```
JavaScript
```

```
let str = "hello"; let arr = [...str]; console.log(arr);
```

Output

```
['h','e','l','l','o']
```

## 7️⃣ Spread with Objects

Spread can copy objects.

```
JavaScript
```

```
let obj1 = {
    name: "Aamir",
    age: 22
}; let obj2 = {...obj1}; console.log(obj2);
```

Output

```
{name:"Aamir", age:22}
```

## 8️⃣ Merging Objects

```
JavaScript
```

```
let obj1 = {a:1, b:2};
let obj2 = {c:3, d:4}; let obj3 = {...obj1, ...obj2}; console.log(obj3);
```

Output

```
{a:1, b:2, c:3, d:4}
```

## 9️⃣ Overwriting Values

If keys are same, **last value wins**.

```
JavaScript
```

```
let obj1 = {name:"Aamir", age:22};
let obj2 = {age:25}; let obj3 = {...obj1, ...obj2}; console.log(obj3);
```

Output

```
{name:"Aamir", age:25}
```

## 🔟 Spread with Objects (Add new property)

```
JavaScript
```

```
let user = {
    name:"Aamir",
    age:22
}; let updatedUser = {
    ...user,
    city:"Delhi"
}; console.log(updatedUser);
```

Output

```
{name:"Aamir", age:22, city:"Delhi"}
```

## 1️⃣1️⃣ Spread with Math Functions

Example with `Math.max`

```
JavaScript
```

```
let arr = [5,8,2,10]; let max = Math.max(...arr); console.log(max);
```

Output

```
10
```

Without spread:

```
Math.max(5,8,2,10)
```

## 1️⃣2️⃣ Spread with Nested Arrays

```
JavaScript



let arr = [[1,2],[3,4]]; let copy = [...arr]; console.log(copy);
```

Output

```
[[1,2],[3,4]]
```

Important:

This is **shallow copy**, not deep copy.

Changing nested data affects both arrays.

## 1️⃣3️⃣ Spread with Objects in Array

```
JavaScript




let users = [
    {name:"Aamir"},
    {name:"Khan"}
]; let copyUsers = [...users]; console.log(copyUsers);
```

Again **shallow copy**.

## 1️⃣4️⃣ Spread vs Rest Operator

Both use `...` but meaning differs.

### Spread → Expand values

```
JavaScript




let arr = [1,2,3];
console.log(...arr);
```

## Rest → Collect values

```
JavaScript




function sum(...nums){
    console.log(nums);
} sum(1,2,3,4);
```

Output

```
[1,2,3,4]
```

## 1️⃣5️⃣ Spread in Real Projects (React Example)

```
JavaScript




let state = {
    name:"Aamir",
    age:22
}; let newState = {
    ...state,
    age:23
};
```

React developers use this **very frequently**.

## 🚀 Spread Operator Summary

| Use | Example |
|---|---|
| Expand array | `...arr` |
| Copy array | `[...arr]` |
| Merge arrays | `[...arr1, ...arr2]` |
| Copy object | `{...obj}` |
| Merge objects | `{...obj1,...obj2}` |
| Function arguments | `sum(...arr)` |

## ⚡ Interview Questions

### 1️⃣ What is spread operator?

Spread operator (`...`) expands iterable values into individual elements.

### 2️⃣ What data types support spread?

- Array
- String
- Object
- Set / Map (iterables)

### 3️⃣ Is spread deep copy?

❌ No
✔ It creates **shallow copy**

### 4️⃣ Spread vs Rest?

| Spread | Rest |
|---|---|
| Expand | Collect |
| Used in arrays/objects | Used in parameters |

## 🔥 Practice Questions

### Q1

```JavaScript
let arr = [1,2,3];
let newArr = [...arr,4,5]; console.log(newArr);
```

### Q2

Find max value.

```JavaScript
let arr = [5,9,2,11];
```

### Q3

Merge objects.

```
JavaScript




let obj1 = {a:1};
let obj2 = {b:2};
```

## Q4

Convert string to array.

```
"javascript"
```

If you want, I can also explain 🔥 **REST operator deeply (very important concept)** including:

- rest parameters
- rest with destructuring
- rest vs spread
- advanced interview patterns.

spread array literals

## Spread in Array Literals (. . .) – Complete Concept

In JavaScript, **spread operator (. . .) inside array literals** is used to **expand elements of an iterable (array, string, set, etc.) into a new array**.

Array literal means:

```
JavaScript



[]
```

When we use `...` inside it, we can **copy, merge, or insert elements easily**.

## 1️⃣ Basic Spread in Array Literal

```
JavaScript



let arr = [1,2,3]; let newArr = [...arr]; console.log(newArr);
```

Output

```
[1,2,3]
```

Here:

```
...arr
```

expands:

```
1,2,3
```

So the new array becomes:

```
[1,2,3]
```

## 2️⃣ Copying an Array

Spread is commonly used to **copy arrays**.

```JavaScript
let arr1 = [10,20,30];
let arr2 = [...arr1]; console.log(arr2);
```

Output

```
[10,20,30]
```

Now modify `arr2`.

```
JavaScript
arr2.push(40); console.log(arr1);
console.log(arr2);
```

Output

```
[10,20,30]
[10,20,30,40]
```

Meaning **original array does not change**.

## 3️⃣ Merging Arrays

Spread can combine arrays.

```
JavaScript
```

```
let arr1 = [1,2,3];
let arr2 = [4,5,6]; let merged = [...arr1, ...arr2]; console.log(merged);
```

Output

```
[1,2,3,4,5,6]
```

## 4️⃣ Adding Elements While Copying

You can insert values anywhere.

### Add at start

```
JavaScript
```

```
let arr = [2,3,4]; let newArr = [1, ...arr]; console.log(newArr);
```

Output

```
[1,2,3,4]
```

## Add at end

```
JavaScript



let arr = [1,2,3]; let newArr = [...arr,4]; console.log(newArr);
```

Output

```
[1,2,3,4]
```

## Add in middle

```
JavaScript



let arr = [1,4,5]; let newArr = [1,2,3,...arr]; console.log(newArr);
```

Output

```
[1,2,3,1,4,5]
```

## 5️⃣ Spread Multiple Arrays

```
JavaScript



let a = [1,2];
let b = [3,4];
let c = [5,6]; let result = [...a, ...b, ...c]; console.log(result);
```

Output

```
[1,2,3,4,5,6]
```

## 6️⃣ Spread with Strings

Strings are iterable, so spread works.

```
JavaScript
```

```
let str = "hello"; let arr = [...str]; console.log(arr);
```

Output

```
['h','e','l','l','o']
```

## 7️⃣ Spread with Function Result

```
JavaScript



function numbers(){
    return [1,2,3];
} let arr = [...numbers(),4,5]; console.log(arr);
```

Output

```
[1,2,3,4,5]
```

## 8️⃣ Spread with Nested Arrays

```
JavaScript



let arr = [[1,2],[3,4]]; let copy = [...arr]; console.log(copy);
```

Output

```
[[1,2],[3,4]]
```

Important:

This is **shallow copy**.

Example:

```
JavaScript



copy[0][0] = 100; console.log(arr);
```

Output

```
[[100,2],[3,4]]
```

Both arrays change because nested arrays share reference.

## 9️⃣ Spread with Conditional Values

```
JavaScript



let isAdmin = true; let arr = [
    "user",
    ...(isAdmin ? ["admin"] : [])
]; console.log(arr);
```

Output

```
["user","admin"]
```

## 🔟 Spread with Math Functions

Example: find max value

```
JavaScript



let arr = [5,8,2,10]; let max = Math.max(...arr); console.log(max);
```

Output

```
10
```

## 1️⃣1️⃣ Spread vs push()

Instead of:

```
JavaScript



let arr1 = [1,2];
let arr2 = [3,4]; arr1.push(...arr2);
```

Result

```
[1,2,3,4]
```

## 1️⃣2️⃣ Real Project Example

Add new item without modifying original array.

```
JavaScript



let cart = ["apple","banana"]; let newCart = [...cart,"orange"]; console.log(newCart);
```

Output

```
["apple","banana","orange"]
```

Used heavily in **React state updates**.

## 🚀 Summary

Spread in array literals is used for:

| Use | Example |
| --- | --- |
| Copy array | `[...arr]` |
| Merge arrays | `[...arr1,...arr2]` |
| Insert elements | `[1,...arr,5]` |
| String to array | `[..."hello"]` |
| Function result | `[...func()]` |

## ⚡ Interview Questions

### 1️⃣ What is spread operator?

`...` expands iterable values into individual elements.

## 2️⃣ Does spread create deep copy?

❌ No
✔ It creates **shallow copy**

## 3️⃣ Can spread work on strings?

✔ Yes

Example:

```
JavaScript



[..."hello"]
```

## 4️⃣ Difference between spread and push?

Spread creates **new array**, push modifies **existing array**.

# 🔥 Practice

## Q1

```
JavaScript



let arr = [2,3,4]; let newArr = [1,...arr,5]; console.log(newArr);
```

## Q2

Merge arrays.

```
JavaScript




let a = [1,2];
let b = [3,4];
```

Expected output

```
[1,2,3,4]
```

## Q3

Convert string to array.

```
"javascript"
```

Expected output

```
['j','a','v','a','s','c','r','i','p','t']
```

If you want, I can also explain **Rest Operator (`...`) deeply** which is **one of the most confusing JavaScript interview topics**, including:

- rest parameters
- rest with objects
- rest with destructuring
- spread vs rest deep comparison.
spread object literals all concept

## Spread Operator with Object Literals (`...`) — Complete Concept

In JavaScript, the **spread operator (`...`)** can also be used inside **object literals `{}`**.

It allows you to:

- Copy objects
- Merge objects
- Update objects
- Add new properties
- Override properties
- Clone objects immutably

This is **very important in React, Node.js, and interviews**.

## 1️⃣ Basic Syntax

```
JavaScript



let newObj = {...oldObj};
```

Example:

```
JavaScript



let obj = {
    name: "Aamir",
    age: 22
}; let copy = {...obj}; console.log(copy);
```

Output

```
{name: "Aamir", age: 22}
```

Spread **copies properties into a new object**.

## 2️⃣ Copying an Object

```
JavaScript



let user = {
    name: "Aamir",
    city: "Delhi"
}; let userCopy = {...user}; console.log(userCopy);
```

Output

```
{name:"Aamir", city:"Delhi"}
```

Now modify copy:

```
JavaScript



userCopy.city = "Mumbai"; console.log(user);
console.log(userCopy);
```

Output

```
{name:"Aamir", city:"Delhi"}
{name:"Aamir", city:"Mumbai"}
```

Original object **does not change**.

## 3️⃣ Adding New Properties

Spread makes it easy to add properties.

```
JavaScript



let user = {
    name: "Aamir",
    age: 22
}; let newUser = {
    ...user,
    city: "Delhi"
}; console.log(newUser);
```

Output

```
{name:"Aamir", age:22, city:"Delhi"}
```

## 4️⃣ Updating Properties

```
JavaScript




let user = {
    name: "Aamir",
    age: 22
}; let updatedUser = {
    ...user,
    age: 25
}; console.log(updatedUser);
```

Output

```
{name:"Aamir", age:25}
```

Spread copies all properties and **new value overwrites old one**.

## 5️⃣ Merging Objects

```
JavaScript




let obj1 = {a:1, b:2};
let obj2 = {c:3, d:4}; let merged = {...obj1, ...obj2}; console.log(merged);
```

Output

```
{a:1, b:2, c:3, d:4}
```

## 6️⃣ Overwriting Properties

If keys are the same, **last value wins**.

```
JavaScript




let obj1 = {name:"Aamir", age:22};
let obj2 = {age:30}; let result = {...obj1, ...obj2}; console.log(result);
```

Output

```
{name:"Aamir", age:30}
```

Because `obj2` overwrites `obj1`.

## 7️⃣ Spread Order Matters

Example:

```
JavaScript



let obj1 = {a:1};
let obj2 = {a:2}; let result = {...obj1, ...obj2}; console.log(result);
```

Output

```
{a:2}
```

Reverse order:

```
JavaScript



let result = {...obj2, ...obj1};
```

Output

```
{a:1}
```

## 8️⃣ Spread with Arrays into Object

Arrays can be spread into objects.

```
JavaScript



let arr = [10,20,30]; let obj = {...arr}; console.log(obj);
```

Output

```
{0:10, 1:20, 2:30}
```

Array indexes become **object keys**.

## 9️⃣ Spread with Strings into Object

```
JavaScript



let str = "abc"; let obj = {...str}; console.log(obj);
```

Output

```
{0:'a',1:'b',2:'c'}
```

## 🔟 Shallow Copy (Important Concept)

Spread creates **shallow copy**, not deep copy.

Example:

```
JavaScript
```

```
let user = {
    name: "Aamir",
    address: {
        city: "Delhi"
    }
}; let copy = {...user}; copy.address.city = "Mumbai"; console.log(user.address.city);
```

Output

```
Mumbai
```

Because nested object **shares reference**.

## 1️⃣1️⃣ Conditional Property Add

```
JavaScript
```

```
let isAdmin = true; let user = {
    name: "Aamir",
    ...(isAdmin && {role:"admin"})
}; console.log(user);
```

Output

```
{name:"Aamir", role:"admin"}
```

## 1️⃣2️⃣ Spread in Real Projects

Example: Updating user object

```
JavaScript



let state = {
    name: "Aamir",
    age: 22,
    city: "Delhi"
}; let newState = {
    ...state,
    age: 23
};
```

Used heavily in **React state updates**.

## 1️⃣3️⃣ Spread with Function Return

```
JavaScript




function getUser(){
    return {name:"Aamir", age:22};
} let user = {
    ...getUser(),
    city:"Delhi"
}; console.log(user);
```

Output

```
{name:"Aamir", age:22, city:"Delhi"}
```

## 1️⃣4️⃣ Spread vs Object.assign()

Old method:

```
JavaScript




let obj2 = Object.assign({}, obj1);
```

Modern way:

```
JavaScript



let obj2 = {...obj1};
```

Spread is **cleaner and easier**.

## 🚀 Summary

| Use | Example |
|---|---|
| Copy object | `{...obj}` |
| Merge objects | `{...obj1,...obj2}` |
| Add property | `{...obj, city:"Delhi"}` |
| Update property | `{...obj, age:25}` |
| Array → object | `{...arr}` |
| String → object | `{...str}` |

## ⚡ Interview Questions

### 1️⃣ What does spread operator do in objects?

It **copies or merges properties into a new object**.

### 2️⃣ Is spread deep copy?

❌ No
✔ It creates **shallow copy**

### 3️⃣ What happens if keys are same?

The **last property overrides the previous one**.

## 4️⃣ Spread vs Object.assign?

Both copy objects, but spread is **simpler syntax**.

## 🔥 Practice Questions

### Q1

```
JavaScript



let obj = {a:1, b:2}; let copy = {...obj}; console.log(copy);
```

### Q2

Merge objects.

```
JavaScript



let obj1 = {name:"Aamir"};
let obj2 = {age:22};
```

Expected output

```
{name:"Aamir", age:22}
```

## Q3

Update property.

```
JavaScript



let user = {name:"Aamir", age:22};
```

Update age to `25`.

## Q4 (Interview Level)

Convert array to object.

```
JavaScript



let arr = [10,20,30];
```

Expected output

```
{0:10,1:20,2:30}
```

If you want, I can also explain 🔥 **Destructuring (array + object) deeply**, which is **one of the most important JavaScript interview topics** after spread and rest.

Rest all concept explain easy word

## Rest Operator (. . .) in JavaScript — Easy Explanation

The **Rest Operator (. . .)** collects multiple values and puts them into **one array or object**.

Think like this:

- **Spread → expand values**
- **Rest → collect values**

## 1️⃣ Rest in Function Parameters

Rest allows a function to accept **unlimited arguments**.

### Syntax

```
JavaScript
```

```
function name(...parameter){
}
```

Example:

```
JavaScript
```

```
function sum(...numbers){
    console.log(numbers);
} sum(1,2,3,4,5);
```

Output

```
[1,2,3,4,5]
```

Explanation:

```
...numbers
```

collects all values into an **array**.

## 2️⃣ Rest with Sum Example

```
JavaScript




function sum(...nums){
    let total = 0;     for(let num of nums){
        total += num;
    }     console.log(total);
} sum(10,20);
sum(10,20,30);
sum(10,20,30,40);
```

Output

```
30
60
100
```

## 3️⃣ Rest with Normal Parameters

Rest must be **last parameter**.

Correct:

```
JavaScript
```

```
function greet(name, ...msg){
    console.log(name);
    console.log(msg);
} greet("Aamir","Hello","How are you");
```

Output

```
Aamir
["Hello","How are you"]
```

Wrong ❌

```
JavaScript
```

```
function test(...a, b){
}
```

Rest **must be last parameter**.

## 4️⃣ Rest with Arrow Function

```
JavaScript




const multiply = (...nums) => {
    console.log(nums);
}; multiply(2,4,6);
```

Output

```
[2,4,6]
```

## 5️⃣ Rest with Array Destructuring

Rest collects **remaining elements**.

Example:

```
JavaScript




let arr = [1,2,3,4,5]; let [a,b,...rest] = arr; console.log(a);
console.log(b);
console.log(rest);
```

Output

```
1
2
[3,4,5]
```

Explanation

```
a = 1
b = 2
rest = [3,4,5]
```

## 6️⃣ Rest with Object Destructuring

```
JavaScript
```

```
let user = {
    name: "Aamir",
    age: 22,
    city: "Delhi",
    country: "India"
}; let {name, ...rest} = user; console.log(name);
console.log(rest);
```

Output

```
Aamir
{age:22, city:"Delhi", country:"India"}
```

Rest collects **remaining properties**.

## 7️⃣ Rest vs Arguments Object

Old JavaScript used:

```
JavaScript



function sum(){
    console.log(arguments);
} sum(1,2,3);
```

But `arguments` is **not a real array**.

Rest operator is better:

```
JavaScript
```

```
function sum(...nums){
    console.log(nums);
}
```

Because `nums` is a **real array**.

## 8️⃣ Rest with Reduce

```
JavaScript
```

```
function sum(...nums){
    return nums.reduce((a,b) => a + b);
} console.log(sum(5,10,15));
```

Output

```
30
```

## 9️⃣ Real Project Example

Example: cart items

```
JavaScript
```

```
function cart(user, ...items){
    console.log(user);
    console.log(items);
} cart("Aamir","apple","banana","mango");
```

Output

```
Aamir
["apple","banana","mango"]
```

## 🔟 Spread vs Rest (Important)

| Spread | Rest |
| --- | --- |
| Expand values | Collect values |
| Used in arrays/objects | Used in parameters/destructuring |

Example:

Spread:

```
JavaScript




let arr = [1,2,3];
console.log(...arr);
```

Output

```
1 2 3
```

Rest:

```
JavaScript




function test(...nums){
    console.log(nums);
} test(1,2,3);
```

Output

```
[1,2,3]
```

## 1️⃣1️⃣ Rest with Nested Destructuring

```
JavaScript
```

```
let arr = [10,20,30,40,50]; let [first,...others] = arr; console.log(first);
console.log(others);
```

Output

```
10
[20,30,40,50]
```

## 1️⃣2️⃣ Rest Must Be Last

Correct:

```
JavaScript



let [a,b,...rest] = [1,2,3,4];
```

Wrong:

```
JavaScript



let [...rest,a] = [1,2,3];
```

## 🚀 Summary

Rest operator is used to:

| Use | Example |
| --- | --- |
| Collect arguments | `function sum(...nums)` |
| Array destructuring | `[a,...rest]` |
| Object destructuring | `{name,...rest}` |
| Unlimited parameters | `sum(1,2,3,4)` |

## ⚡ Interview Questions

### 1️⃣ What is Rest Operator?

Rest operator (`...`) collects multiple values into **one array or object**.

## 2️⃣ Where can rest be used?

- Function parameters
- Array destructuring
- Object destructuring

## 3️⃣ Rest must appear where?

✔ **Last parameter**

## 4️⃣ Rest vs Spread?

Spread **expands**, Rest **collects**.

# 🔥 Practice Questions

## Q1

```
JavaScript



function sum(...nums){
 console.log(nums);
} sum(1,2,3);
```

Output?

## Q2

```
JavaScript




let arr = [10,20,30,40]; let [a,...b] = arr; console.log(a);
console.log(b);
```

## Q3

```
JavaScript




let user = {
 name:"Aamir",
 age:22,
 city:"Delhi"
}; let {name,...rest} = user; console.log(rest);
```

If you want, I can also explain 🔥 **JavaScript Destructuring (Array + Object)** which is one of the **MOST IMPORTANT JS interview concepts after spread and rest.**

spread in easy word

## Spread Operator (. . .) — In Very Easy Words

**Spread** means **"फैलाना / expand करना"**.

The **spread operator (. . .)** takes **values from an array, object, or string and spreads them into individual elements**.

Think like this:

```
Spread = Break things apart
```

## 1️⃣ Basic Example

```
JavaScript
```

```
let arr = [1,2,3]; console.log(...arr);
```

Output

```
1 2 3
```

Explanation

```
...arr → 1,2,3
```

Spread **opens the array**.

## 2️⃣ Spread with Arrays

### Copy an array

```
JavaScript



let arr1 = [10,20,30]; let arr2 = [...arr1]; console.log(arr2);
```

Output

```
[10,20,30]
```

It creates a **new copy**.

## 3️⃣ Merge Arrays

```
JavaScript



let a = [1,2];
let b = [3,4]; let c = [...a,...b]; console.log(c);
```

Output

```
[1,2,3,4]
```

Spread **joins arrays**.

## 4️⃣ Add Elements

```
JavaScript
```

```
let arr = [2,3,4]; let newArr = [1,...arr,5]; console.log(newArr);
```

Output

```
[1,2,3,4,5]
```

Spread **inserts elements easily**.

## 5️⃣ Spread with Objects

```
JavaScript



let user = {
 name:"Aamir",
 age:22
}; let copy = {...user}; console.log(copy);
```

Output

```
{name:"Aamir", age:22}
```

Spread **copies object properties**.

## 6️⃣ Merge Objects

```
JavaScript



let obj1 = {a:1};
let obj2 = {b:2}; let obj3 = {...obj1,...obj2}; console.log(obj3);
```

Output

```
{a:1,b:2}
```

## 7️⃣ Spread with Strings

```
JavaScript
```

```
let str = "hello"; let arr = [...str]; console.log(arr);
```

Output

```
["h","e","l","l","o"]
```

## 8️⃣ Spread with Functions

```
JavaScript



function sum(a,b,c){
 console.log(a+b+c);
} let nums = [10,20,30]; sum(...nums);
```

Output

```
60
```

Spread **sends array values as arguments**.

## 🔑 Simple Definition

Spread operator (`...`) **expands values from array, object, or string**.

## ⚡ Spread vs Rest (Easy)

Spread **expands** values.

```
...arr → 1,2,3
```

Rest **collects** values.

```
function sum(...nums)
```

## 🚀 Real Life Example

Imagine a **bag of apples**.

```
bag = [🍎🍎🍎]
```

Spread opens the bag:

```
...bag → 🍎 🍎 🍎
```

## 🔥 Most Common Uses

1. Copy array
2. Merge arrays
3. Copy object
4. Merge objects
5. Pass array to function

If you want, I can also show 🔥 **10 very important spread interview questions** that every **JavaScript developer is asked**.

destructuring all concept explain

## JavaScript Destructuring — Complete Concept (Easy Words)

**Destructuring** means **extracting values from arrays or objects and storing them into variables easily**.

Instead of writing many lines like:

```
JavaScript




let arr = [10,20,30]; let a = arr[0];
let b = arr[1];
let c = arr[2];
```

We can use **destructuring**:

```
JavaScript




let [a,b,c] = [10,20,30]; console.log(a,b,c);
```

Output

```
10 20 30
```

So destructuring = **shortcut for extracting values**.

## 1️⃣ Array Destructuring

Extract values from an array.

### Example

```
JavaScript



let arr = [1,2,3]; let [a,b,c] = arr; console.log(a);
console.log(b);
console.log(c);
```

Output

```
1
2
3
```

## 2️⃣ Skipping Values

You can skip elements using comma.

```
JavaScript
```

```
let arr = [10,20,30,40]; let [a,,c] = arr; console.log(a);
console.log(c);
```

Output

```
10
30
```

## 3️⃣ Default Values

If value doesn't exist, default value is used.

```
JavaScript
```

```
let arr = [5,10]; let [a,b,c=20] = arr; console.log(c);
```

Output

```
20
```

## 4️⃣ Swap Variables (Very Popular Interview Question)

```
JavaScript
```

```
let a = 10;
let b = 20; [a,b] = [b,a]; console.log(a,b);
```

Output

```
20 10
```

No temporary variable needed.

## 5️⃣ Rest Operator with Destructuring

Collect remaining values.

```
JavaScript
```

```
let arr = [1,2,3,4,5]; let [a,b,...rest] = arr; console.log(a);
console.log(b);
console.log(rest);
```

Output

```
1
2
[3,4,5]
```

## 6️⃣ Nested Array Destructuring

```
JavaScript
```

```
let arr = [1,2,[3,4]]; let [a,b,[c,d]] = arr; console.log(c,d);
```

Output

```
3 4
```

## 7️⃣ Object Destructuring

Extract values from objects.

```
JavaScript



let user = {
    name: "Aamir",
    age: 22,
    city: "Delhi"
}; let {name,age} = user; console.log(name);
console.log(age);
```

Output

```
Aamir
22
```

Important:
Object destructuring uses **property names**.

## 8️⃣ Rename Variables

```
JavaScript
```

```
let user = {
    name: "Aamir",
    age: 22
}; let {name:username, age:userAge} = user; console.log(username);
console.log(userAge);
```

Output

```
Aamir
22
```

## 9️⃣ Default Values in Object

```
JavaScript
```

```
let user = {
    name:"Aamir"
}; let {name,age=25} = user; console.log(age);
```

Output

```
25
```

## 🔟 Rest Operator with Objects

Collect remaining properties.

```
JavaScript
```

```
let user = {
    name:"Aamir",
    age:22,
    city:"Delhi"
}; let {name,...rest} = user; console.log(name);
console.log(rest);
```

Output

```
Aamir
{age:22, city:"Delhi"}
```

## 1️⃣1️⃣ Nested Object Destructuring

```
JavaScript




let user = {
    name:"Aamir",
    address:{
        city:"Delhi",
        country:"India"
    }
}; let {address:{city}} = user; console.log(city);
```

Output

```
Delhi
```

## 1️⃣2️⃣ Destructuring in Function Parameters

Very common in React and APIs.

```
JavaScript
```

```
function printUser({name,age}){
    console.log(name,age);
} let user = {
    name:"Aamir",
    age:22
}; printUser(user);
```

Output

```
Aamir 22
```

## 1️⃣3️⃣ Destructuring with Arrays of Objects

Example:

```
JavaScript
```

```
let users = [
    {name:"Aamir",age:22},
    {name:"Khan",age:25}
]; let [{name}, {age}] = users; console.log(name);
console.log(age);
```

Output

```
Aamir
25
```

## 🚀 Array vs Object Destructuring

| Array | Object |
| --- | --- |
| Use `[]` | Use `{}` |
| Based on position | Based on property name |

Example:

```JavaScript
let [a,b] = [1,2];
let {name} = {name:"Aamir"};
```

## 🔥 Real World Example

Array destructuring in loops:

```
JavaScript
```

```
let arr = [[1,2],[3,4]]; for(let [a,b] of arr){
    console.log(a,b);
}
```

Output

```
1 2
3 4
```

## ⚡ Interview Questions

### 1️⃣ What is destructuring?

Destructuring is a **JavaScript feature used to extract values from arrays or objects into variables**.

### 2️⃣ Types of destructuring

1. Array destructuring
2. Object destructuring

### 3️⃣ Can we give default values?

✔ Yes.

### 4️⃣ Can we rename variables?

✔ Yes.

## 5️⃣ Can we use rest operator?

✔ Yes.

# 🔥 Practice Questions

## Q1

```
JavaScript



let arr = [10,20,30]; let [a,b,c] = arr; console.log(a,b,c);
```

## Q2

Swap variables.

```
JavaScript



let x = 5;
let y = 10;
```

## Q3

```
JavaScript




let user = {
 name:"Aamir",
 age:22,
 city:"Delhi"
}; let {name,...rest} = user; console.log(rest);
```

If you want, I can also explain 🔥 **15 MOST IMPORTANT JavaScript Destructuring Interview Questions with solutions** (very helpful for React and Node.js interviews).

destructuring objects

## JavaScript Object Destructuring — Easy & Complete Explanation

**Object destructuring** means **extracting values from an object and storing them into variables easily**.

Normally we access object values like this:

```
JavaScript




let user = {
    name: "Aamir",
    age: 22,
    city: "Delhi"
}; let name = user.name;
let age = user.age;
```

But with **destructuring**, we can write:

```
JavaScript
```

```
let {name, age} = user; console.log(name);
console.log(age);
```

Output

```
Aamir
22
```

So destructuring = **shortcut to get object values**.

## 1️⃣ Basic Object Destructuring

```
JavaScript
```

```
let user = {
    name: "Aamir",
    age: 22,
    city: "Delhi"
}; let {name, age, city} = user; console.log(name);
console.log(age);
console.log(city);
```

Output

```
Aamir
22
Delhi
```

Important rule:

Variable name must match **object property name**.

## 2️⃣ Store in Different Variable Name (Rename)

Sometimes we want different variable names.

```
JavaScript
```

```
let user = {
    name: "Aamir",
    age: 22
}; let {name: username, age: userAge} = user; console.log(username);
console.log(userAge);
```

Output

```
Aamir
22
```

Explanation

```
name → username
age → userAge
```

## 3️⃣ Default Values

If a property does not exist, default value will be used.

```
JavaScript
```

```
let user = {
    name: "Aamir"
}; let {name, age = 25} = user; console.log(age);
```

Output

```
25
```

Because `age` was not present.

## 4️⃣ Destructuring with Rest Operator

Rest collects remaining properties.

```
JavaScript
```

```
let user = {
    name: "Aamir",
    age: 22,
    city: "Delhi",
    country: "India"
}; let {name, ...rest} = user; console.log(name);
console.log(rest);
```

Output

```
Aamir
{age:22, city:"Delhi", country:"India"}
```

## 5️⃣ Nested Object Destructuring

Objects can contain objects.

```
JavaScript
```

```
let user = {
    name: "Aamir",
    address: {
        city: "Delhi",
        country: "India"
    }
}; let {address:{city}} = user; console.log(city);
```

## Output

```
Delhi
```

## 6️⃣ Multiple Nested Values

```
JavaScript
```

```
let user = {
    name: "Aamir",
    address: {
        city: "Delhi",
        country: "India"
    }
}; let {address:{city,country}} = user; console.log(city);
console.log(country);
```

Output

```
Delhi
India
```

## 7️⃣ Destructuring in Function Parameters

Very common in **React and APIs**.

```
JavaScript
```

```
function printUser({name,age}){
    console.log(name,age);
} let user = {
    name:"Aamir",
    age:22
}; printUser(user);
```

Output

```
Aamir 22
```

## 8️⃣ Destructuring Array of Objects

Example:

```
JavaScript
```

```
let students = [
    {name:"Aamir", marks:95},
    {name:"Khan", marks:90}
]; let [{name}, {marks}] = students; console.log(name);
console.log(marks);
```

Output

```
Aamir
90
```

## 9️⃣ Combining Spread with Destructuring

```
JavaScript
```

```
let user = {
    name:"Aamir",
    age:22,
    city:"Delhi"
}; let {name,...details} = user; console.log(details);
```

Output

```
{age:22, city:"Delhi"}
```

## 🔟 Avoid Error with Default Object

Sometimes object may be undefined.

```
JavaScript




function showUser({name} = {}){
    console.log(name);
} showUser();
```

Without `= {}` it may cause error.

## 🚀 Real Project Example

API response example.

```
JavaScript




let response = {
    status: "success",
    data: {
        username: "aamir",
        followers: 1200
    }
}; let {data:{username,followers}} = response; console.log(username);
console.log(followers);
```

Output

```
aamir
1200
```

## ⚡ Important Rules

1️⃣ Object destructuring uses `{}`

```
let {name} = user
```

2️⃣ Variable name must match property name.

3️⃣ Use `:` to rename variables.

4️⃣ Use `=` for default values.

5️⃣ Use `...` for rest properties.

## 🔥 Most Common Interview Question

### What is Object Destructuring?

Object destructuring is a **JavaScript feature that extracts properties from an object and assigns them to variables using `{}` syntax**.

# 🔥 Practice Questions

## Q1

```
JavaScript




let user = {
 name:"Aamir",
 age:22
}; let {name,age} = user; console.log(name);
```

Output?

## Q2

```
JavaScript




let user = {
 name:"Aamir"
}; let {name,city="Delhi"} = user; console.log(city);
```

## Q3

```
JavaScript




let user = {
 name:"Aamir",
 age:22,
 city:"Delhi"
}; let {name,...rest} = user; console.log(rest);
```

If you want, I can also show **10 tricky destructuring interview questions** that confuse most JavaScript developers.